

Qdrant Vector Database cho AI

Từ nền tảng vector search đến RAG, hybrid search và memory cho agent

Mục tiêu: hiểu sâu bản chất Qdrant, dùng thành thạo Python client, thiết kế collection đúng, tối ưu search/filter/index, và ứng dụng vào các mô hình AI thực tế.

Mục lục

1 Qdrant là gì và vì sao quan trọng trong AI?	4
1.1 Qdrant giải quyết bài toán gì?	4
1.2 Luồng tổng quát trong một hệ AI dùng Qdrant	4
2 Từ embedding đến vector search	5
2.1 Embedding là gì?	5
2.2 Distance metric: Cosine, Dot, Euclid, Manhattan	5
3 Cài đặt và chạy Qdrant	5
3.1 Cách 1: Chạy bằng Docker	5
3.2 Cách 2: Local mode trong Python	6
3.3 Cách 3: Kết nối Qdrant server/cloud	6
3.3.1 Giải thích tham số kết nối	6
4 Data model: collection, point, vector, payload	7
4.1 Collection	7
4.2 Point	7
4.3 Payload	7
5 Tạo collection đầu tiên	8
5.1 Giải thích tham số	8
6 Embedding thật và upsert dữ liệu	8
6.1 Cài thư viện	8
6.2 Code end-to-end nhỏ nhất	9
6.3 Input/output trực quan	9
6.4 Giải thích các phần quan trọng	10
7 Query/search cơ bản	10
7.1 Giải thích tham số query	10
7.2 Output thường gặp	10
8 Filtering: tìm đúng ngữ cảnh, đúng quyền, đúng nguồn	11
8.1 Must, should, must_not	11
8.2 Range filter	11
8.3 Nested metadata và array	12
9 Payload index: filter nhanh hơn và query planner tốt hơn	12
9.1 Chọn field nào để index?	12

10 Scroll, retrieve, update, delete	12
10.1 Retrieve theo id	13
10.2 Scroll để duyệt dữ liệu	13
10.3 Update payload	13
10.4 Delete theo id hoặc filter	13
11 Named vectors: một point, nhiều cách biểu diễn	14
12 HNSW: bản chất index tìm kiếm gần đúng	15
12.1 Các tham số HNSW hay gặp	15
12.2 Tạo collection với HNSW config	15
12.3 Search với search_params	16
13 Quantization: giảm RAM, tăng tốc, chấp nhận xấp xỉ	16
13.1 Giải thích tham số quantization	16
14 Xây project RAG cơ bản bằng Qdrant	17
14.1 Cấu trúc project	17
14.2 docker-compose.yml	17
14.3 requirements.txt	17
14.4 src/config.py	17
14.5 src/chunking.py	18
14.5.1 Giải thích chunking	18
14.6 src/ingest.py	18
14.7 src/search.py	19
14.8 src/rag.py	21
14.9 Luồng input/output của RAG	21
15 Hybrid search: dense + sparse + fusion	22
15.1 Khi nào cần hybrid?	22
15.2 Universal Query API với RRF	22
15.3 RRF trực quan	22
16 Multivector và ColBERT reranking	23
16.1 Khi nào dùng multivector?	23
17 Multitenancy và phân quyền retrieval	23
18 Ứng dụng Qdrant cho memory của agent	24
18.1 Memory nên lưu gì?	24
18.2 Code memory service tối giản	24
18.3 Cơ chế memory retrieval trong agent	26

19Đánh giá chất lượng retrieval	26
19.1Các metric quan trọng	26
19.2Code đánh giá Recall@k đơn giản	26
20Tối ưu production checklist	27
21Những lỗi phổ biến khi mới dùng Qdrant	27
22Bài tập vận dụng	27
22.1Mức cơ bản	27
22.2Mức trung bình	28
22.3Mức nâng cao	28
22.4Đề tài project cuối khóa	28
23Lộ trình luyện tập 14 ngày	28
24Cheat sheet lệnh Qdrant/Python hay dùng	29
25Tài liệu tham khảo chính	29

1 Qdrant là gì và vì sao quan trọng trong AI?

Qdrant là một vector database/vector search engine dùng để lưu, quản lý và tìm kiếm các embedding. Trong AI hiện đại, nhiều loại dữ liệu như văn bản, ảnh, âm thanh, đoạn code, hồ sơ người dùng hoặc memory của agent thường được biến thành vector. Các vector gần nhau trong không gian embedding thường biểu diễn các đối tượng có ý nghĩa gần nhau.

Cách nghĩ đơn giản

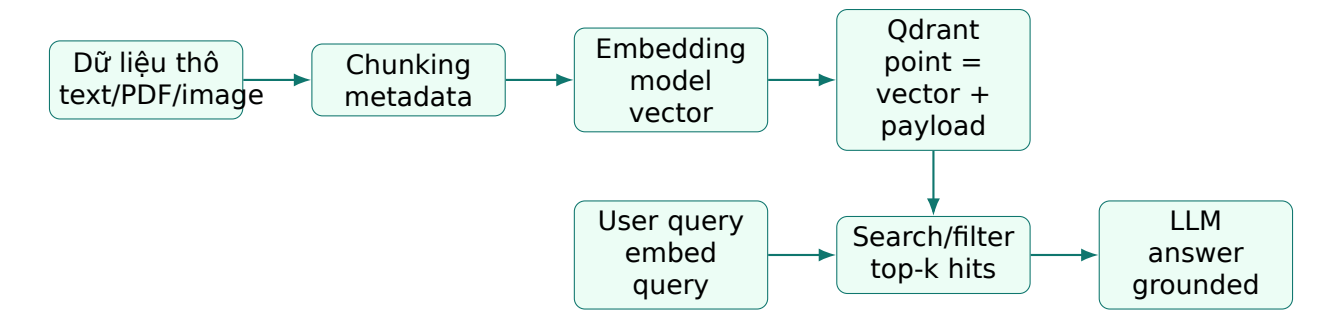
Một database truyền thống thường hỏi: “hàng nào có category = skin và year = 2025?” Qdrant thường hỏi: “đoạn nào gần nghĩa nhất với câu hỏi của người dùng, nhưng chỉ trong tài liệu thuộc tenant này, ngôn ngữ này, quyền truy cập này?”

Theo tài liệu chính thức, Qdrant tổ chức dữ liệu quanh **collection**; mỗi **point** có id, vector và payload metadata. Qdrant hỗ trợ dense vector, sparse vector, named vector, multivector, filtering, payload index, HNSW, quantization, snapshot và các cơ chế triển khai production.

1.1 Qdrant giải quyết bài toán gì?

Bài toán AI	Vai trò của Qdrant
RAG chatbot	Lưu chunk tài liệu, truy xuất top-k đoạn liên quan, đưa vào prompt cho LLM.
Semantic search	Tìm theo ý nghĩa, không cần trùng từ khóa tuyệt đối.
Hybrid search	Kết hợp dense vector search với sparse/keyword search để vừa hiểu nghĩa vừa giữ chính xác thuật ngữ.
Agent memory	Lưu ký ức dài hạn, truy hồi ký ức theo ngữ cảnh hội thoại hiện tại.
Recommendation	Tìm sản phẩm/người dùng/nội dung tương tự dựa trên embedding hành vi hoặc mô tả.
Multimodal search	Lưu vector ảnh + text bằng named vectors, tìm kiếm chéo modality.
Deduplication/clustering	Phát hiện nội dung gần trùng, gom nhóm văn bản/ảnh tương tự.

1.2 Luồng tổng quát trong một hệ AI dùng Qdrant



Input: tài liệu + câu hỏi. Output: các hit liên quan, sau đó LLM tạo câu trả lời có căn cứ.

2 Từ embedding đến vector search

2.1 Embedding là gì?

Embedding là danh sách số thực biểu diễn ý nghĩa của dữ liệu. Ví dụ một câu tiếng Việt có thể được biến thành vector 384 chiều:

```
1 Input text:  
2 "Qdrant dùng để lưu embedding và tìm kiếm ngữ nghĩa."  
3  
4 Embedding output, rút gọn:  
5 [0.012, -0.084, 0.233, ..., 0.019] # 384 số
```

Hai câu gần nghĩa thường có vector gần nhau. Vector DB như Qdrant không tự hiểu tiếng Việt theo kiểu con người; nó dựa vào embedding model. Vì vậy chất lượng retrieval phụ thuộc rất lớn vào model embedding, chunking, metadata và cách search.

2.2 Distance metric: Cosine, Dot, Euclid, Manhattan

Khi tạo collection, bạn phải chọn metric. Metric phải phù hợp với cách embedding model được huấn luyện.

Metric	Ý nghĩa	Khi dùng
Cosine	Đo góc giữa 2 vector; ít phụ thuộc độ dài vector.	Phổ biến nhất cho sentence embeddings/RAG. Qdrant tự normalize vector khi dùng cosine.
Dot	Tích vô hướng; score cao nếu hướng và độ lớn phù hợp.	Dùng khi model được train để tối ưu dot product.
Euclid	Khoảng cách hình học L2.	Dữ liệu số, không gian metric rõ ràng.
Manhattan	Khoảng cách L1.	Một số bài toán vector đặc thù.

Sai metric có thể làm retrieval tệ

Nếu model khuyến nghị cosine mà bạn dùng Euclid, kết quả có thể lệch. Khi không chắc, hãy xem card/documentation của embedding model. Với các model sentence-transformers phổ biến, cosine thường là lựa chọn an toàn.

3 Cài đặt và chạy Qdrant

3.1 Cách 1: Chạy bằng Docker

```
1 docker pull qdrant/qdrant  
2  
3 docker run -p 6333:6333 -p 6334:6334 \  
4   -v "$(pwd)/qdrant_storage:/qdrant/storage:z" \  
5   qdrant/qdrant
```

Sau khi chạy:

- REST API: `http://localhost:6333`
- Web UI: `http://localhost:6333/dashboard`
- gRPC API: `localhost:6334`

3.2 Cách 2: Local mode trong Python

Local mode rất tiện để học, test, Colab hoặc unit test. Khi scale thật thì chuyển sang server/cloud.

```
1 from qdrant_client import QdrantClient
2
3 # Bilen mat khi ket thuc chuong trinh
4 client = QdrantClient(":memory:")
5
6 # Luu tren dia de lan sau doc lai
7 client = QdrantClient(path="./qdrant_local")
```

3.3 Cách 3: Kết nối Qdrant server/cloud

```
1 import os
2 from qdrant_client import QdrantClient
3
4 client = QdrantClient(url="http://localhost:6333")
5
6 # Neu dung Qdrant Cloud
7 cloud_client = QdrantClient(
8     url=os.getenv("QDRANT_URL"),
9     api_key=os.getenv("QDRANT_API_KEY"),
10 )
11
12 # gRPC thuong nhanh hon cho throughput lon
13 fast_client = QdrantClient(
14     host="localhost",
15     port=6333,
16     grpc_port=6334,
17     prefer_grpc=True,
18 )
```

3.3.1 Giải thích tham số kết nối

Tham số	Vai trò
url	Endpoint REST của Qdrant, ví dụ <code>http://localhost:6333</code> .
api_key	Khóa xác thực, đặc biệt khi dùng Qdrant Cloud hoặc server bật bảo mật.
host, port	Cách viết tách host/port thay vì truyền nguyên URL.
grpc_port	Port gRPC, mặc định 6334.
prefer_grpc=True	Ưu tiên gRPC cho các method custom của client, hữu ích khi upload/search nhiều.

timeout	Số giây chờ request. Dữ liệu lớn nên tăng timeout cho upload/index.
path	Local mode có lưu xuống đĩa.
" :memory: "	Local mode chỉ lưu trong RAM, phù hợp test nhanh.

4 Data model: collection, point, vector, payload

4.1 Collection

Collection là một không gian tìm kiếm. Nếu bạn lưu tài liệu RAG về da liễu, có thể tạo collection `derma_docs`. Nếu làm memory agent, có thể tạo `agent_memories`.

Quy tắc thiết kế collection

Tất cả vector trong cùng một vector field phải có cùng số chiều và cùng metric. Nếu cần lưu nhiều loại vector cho một point, dùng named vectors: ví dụ `dense`, `sparse`, `image`, `title`.

4.2 Point

Một point là bản ghi cơ bản trong Qdrant:

```
1 {
2   "id": "a3a9f8e2-...",
3   "vector": [0.12, -0.04, 0.91, "..."],
4   "payload": {
5     "text": "Qdrant là vector database...",
6     "source": "rag_notes.md",
7     "chunk_id": 7,
8     "tenant_id": "user_001",
9     "tags": ["rag", "qdrant"]
10  }
11 }
```

4.3 Payload

Payload là metadata dạng JSON. Payload không thay thế database quan hệ, nhưng rất quan trọng để filter, phân quyền, group, trace nguồn tài liệu.

Payload field	Nên dùng để làm gì
text	Nội dung chunk để đưa vào prompt hoặc hiển thị kết quả search.
source	Tên file/URL/tài liệu gốc.
page	Trang PDF hoặc vị trí tài liệu.
tenant_id	Tách dữ liệu theo người dùng/tổ chức.
doc_type	Loại tài liệu: policy, transcript, paper, note.
created_at	Lọc theo thời gian, ưu tiên tài liệu mới.
tags	Lọc theo chủ đề.
access_roles	Phân quyền truy cập ở tầng retrieval.

5 Tạo collection đầu tiên

Ví dụ dùng embedding model `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, output 384 chiều, phù hợp học và hỗ trợ đa ngôn ngữ trong đó có tiếng Việt.

```
1 from qdrant_client import QdrantClient, models
2
3 COLLECTION = "ai_docs"
4 DIM = 384
5
6 client = QdrantClient(url="http://localhost:6333")
7
8 if client.collection_exists(COLLECTION):
9     client.delete_collection(COLLECTION)
10
11 client.create_collection(
12     collection_name=COLLECTION,
13     vectors_config=models.VectorParams(
14         size=DIM,
15         distance=models.Distance.COSINE,
16     ),
17     on_disk_payload=True,
18 )
```

5.1 Giải thích tham số

Tham số	Giải thích
<code>collection_name</code>	Tên collection. Nên đặt theo domain: <code>ai_docs</code> , <code>agent_memories</code> .
<code>vectors_config</code>	Cấu hình vector chính. Nếu chỉ có một vector field, dùng <code>VectorParams</code> . Nếu nhiều vector field, truyền dictionary.
<code>size</code>	Số chiều vector. Phải đúng với embedding model. Sai size sẽ lỗi khi upsert.
<code>distance</code>	Metric so khớp vector: Cosine, Dot, Euclid, Manhattan.
<code>on_disk_payload</code>	Nếu true, payload không nhất thiết giữ toàn bộ trong RAM. Payload field đã index vẫn được giữ để filter nhanh.

Không đoán DIM

Bạn phải biết embedding model trả về bao nhiêu chiều. Ví dụ MiniLM thường 384, bge-base 768, text-embedding-3-large 3072. Tạo collection sai dimension là lỗi phổ biến nhất khi mới học Qdrant.

6 Embedding thật và upsert dữ liệu

6.1 Cài thư viện

```
1 pip install qdrant-client sentence-transformers numpy
```

6.2 Code end-to-end nhỏ nhất

```
1 from uuid import uuid4
2 from qdrant_client import QdrantClient, models
3 from sentence_transformers import SentenceTransformer
4
5 COLLECTION = "ai_docs"
6 MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
7
8 texts = [
9     "Qdrant lưu trữ vector embedding và hỗ trợ tìm kiếm ngữ nghĩa.",
10    "RAG kết hợp retrieval từ vector database với mô hình ngôn ngữ lớn.",
11    "Payload index giúp filter metadata nhanh hơn trong Qdrant.",
12    "HNSW là cấu trúc đồ thị dùng cho approximate nearest neighbor search.",
13]
14
15 model = SentenceTransformer(MODEL_NAME)
16 embeddings = model.encode(texts, normalize_embeddings=True).tolist()
17
18 client = QdrantClient(url="http://localhost:6333")
19
20 points = []
21 for idx, (text, vector) in enumerate(zip(texts, embeddings)):
22     points.append(
23         models.PointStruct(
24             id=str(uuid4()),
25             vector=vector,
26             payload={
27                 "text": text,
28                 "source": "lesson_demo",
29                 "chunk_id": idx,
30                 "doc_type": "qdrant_note",
31                 "lang": "vi",
32             },
33         )
34     )
35
36 client.upsert(
37     collection_name=COLLECTION,
38     points=points,
39     wait=True,
40 )
```

6.3 Input/output trực quan

```
1 Input texts
2 -> sentence-transformers model
3 -> embeddings: List[List[float]], shape = (4, 384)
4 -> PointStruct(id, vector, payload)
5 -> Qdrant collection: ai_docs
6
7 Output của upsert
8 -> UpdateResult(status='completed', operation_id=...)
```

6.4 Giải thích các phần quan trọng

- `uuid4()`: tạo id duy nhất cho point. Dùng UUID an toàn hơn khi dữ liệu đến từ nhiều file hoặc nhiều người dùng.
- `normalize_embeddings=True`: chuẩn hóa vector về độ dài 1. Với cosine, việc này thường giúp score ổn định. Qdrant cũng normalize vector khi dùng cosine, nhưng normalize từ model giúp bạn dễ debug và so sánh ngoài Qdrant.
- `PointStruct`: object gồm id, vector, payload. Đây là đơn vị ghi vào Qdrant.
- `wait=True`: client chờ đến khi operation được apply. Khi ingest rất lớn có thể dùng `wait=False` để nhanh hơn nhưng cần chấp nhận eventual visibility.

7 Query/search cơ bản

```

1 query = "Qdrant dùng HNSW để làm gì?"
2 query_vector = model.encode(query, normalize_embeddings=True).tolist()
3
4 result = client.query_points(
5     collection_name=COLLECTION,
6     query=query_vector,
7     limit=3,
8     with_payload=True,
9     with_vectors=False,
10 )
11
12 for hit in result.points:
13     print("score:", round(hit.score, 4))
14     print("text :", hit.payload["text"])
15     print("---")

```

7.1 Giải thích tham số query

Tham số	Vai trò
<code>collection_name</code>	Collection cần tìm.
<code>query</code>	Vector truy vấn, id point, sparse vector, multivector hoặc query object tùy mode. Với RAG cơ bản, đây là embedding của câu hỏi.
<code>limit</code>	Số kết quả top-k trả về. RAG thường dùng 3-10; reranking có thể retrieve 30-200 rồi lọc lại.
<code>with_payload=True</code>	Trả metadata/text kèm kết quả. RAG cần bật để lấy chunk text.
<code>with_vectors=False</code>	Không trả vector để tiết kiệm băng thông. Chỉ bật khi debug hoặc cần xử lý tiếp.
<code>score_threshold</code>	Ngưỡng similarity. Dùng để tránh đưa context quá xa nghĩa vào LLM.

7.2 Output thường gặp

```

1 score: 0.8123
2 text : HNSW là cấu trúc đồ thị dùng cho approximate nearest neighbor search.
3 ---

```

```

4 score: 0.7011
5 text : Qdrant lưu trữ vector embedding và hỗ trợ tìm kiếm ngữ nghĩa.
6 ---

```

8 Filtering: tìm đúng ngữ cảnh, đúng quyền, đúng nguồn

Embedding không biểu diễn tốt mọi điều kiện nghiệp vụ. Ví dụ: người dùng A không được xem dữ liệu của người dùng B; tài liệu nội bộ chỉ cho role admin; chỉ tìm trong file policy.pdf. Những điều kiện đó nên nằm trong payload filter.

```

1 flt = models.Filter(
2     must=[
3         models.FieldCondition(
4             key="doc_type",
5             match=models.MatchValue(value="qdrant_note"),
6         ),
7         models.FieldCondition(
8             key="lang",
9             match=models.MatchValue(value="vi"),
10        ),
11    ]
12 )
13
14 result = client.query_points(
15     collection_name=COLLECTION,
16     query=query_vector,
17     query_filter=flt,
18     limit=3,
19     with_payload=True,
20 )

```

8.1 Must, should, must_not

Clause	Ý nghĩa
must	Tất cả điều kiện phải đúng. Tương tự AND.
should	Nên khớp một hoặc nhiều điều kiện. Tương tự OR mềm/OR logic tùy cấu hình.
must_not	Điều kiện không được đúng. Tương tự NOT.

8.2 Range filter

```

1 flt = models.Filter(
2     must=[
3         models.FieldCondition(
4             key="chunk_id",
5             range=models.Range(gte=1, lte=3),
6         )
7     ]
8 )

```

8.3 Nested metadata và array

Payload có thể chứa object hoặc list. Ví dụ:

```
1 {  
2   "metadata": {  
3     "source": "paper.pdf",  
4     "page": 12  
5   },  
6   "access_roles": ["student", "admin"]  
7 }
```

Bạn có thể filter nested field bằng dot notation, ví dụ metadata.source hoặc metadata.page.

9 Payload index: filter nhanh hơn và query planner tốt hơn

Nếu collection nhỏ vài nghìn point, filter không index vẫn có thể chạy. Nhưng với hàng trăm nghìn hoặc hàng triệu point, hãy tạo payload index cho các field hay filter.

```
1 client.create_payload_index(  
2   collection_name=COLLECTION,  
3   field_name="doc_type",  
4   field_schema=models.PayloadSchemaType.KEYWORD,  
5 )  
6  
7 client.create_payload_index(  
8   collection_name=COLLECTION,  
9   field_name="chunk_id",  
10  field_schema=models.PayloadSchemaType.INTEGER,  
11 )
```

9.1 Chọn field nào để index?

- Nên index: tenant_id, user_id, source, doc_type, created_at, access_roles, category.
- Không nhất thiết index: field chỉ dùng để hiển thị như text, summary, raw_html.
- Ưu tiên field có khả năng thu hẹp kết quả mạnh. Ví dụ tenant_id thường quan trọng hơn lang nếu hệ thống chỉ có 2 ngôn ngữ.

Index trước khi ingest nếu biết trước filter

Trong production, nếu đã biết field nào sẽ filter, nên tạo payload index trước khi upload lượng dữ liệu lớn để tránh rebuild tốn tài nguyên.

10 Scroll, retrieve, update, delete

10.1 Retrieve theo id

```
1 records = client.retrieve(  
2     collection_name=COLLECTION,  
3     ids=["point-id-1", "point-id-2"],  
4     with_payload=True,  
5     with_vectors=False,  
6 )
```

10.2 Scroll để duyệt dữ liệu

Scroll không tính similarity. Nó dùng để duyệt toàn bộ point hoặc các point khớp filter, tương tự pagination.

```
1 next_offset = None  
2 while True:  
3     records, next_offset = client.scroll(  
4         collection_name=COLLECTION,  
5         scroll_filter=models.Filter(  
6             must=[  
7                 models.FieldCondition(  
8                     key="doc_type",  
9                     match=models.MatchValue(value="qdrant_note"),  
10                )  
11            ]  
12        ),  
13        limit=100,  
14        offset=next_offset,  
15        with_payload=True,  
16        with_vectors=False,  
17    )  
18  
19    for r in records:  
20        print(r.id, r.payload.get("text"))  
21  
22    if next_offset is None:  
23        break
```

10.3 Update payload

```
1 client.set_payload(  
2     collection_name=COLLECTION,  
3     payload={"reviewed": True},  
4     points=["point-id-1"],  
5     wait=True,  
6 )
```

10.4 Delete theo id hoặc filter

```
1 # Xóa theo id  
2 client.delete(  
3     collection_name=COLLECTION,  
4     points=["point-id-1"],  
5     wait=True,  
6 )
```

```

3     collection_name=COLLECTION,
4     points_selector=["point-id-1"],
5     wait=True,
6 )
7
8 # Xóa theo filter
9 client.delete(
10     collection_name=COLLECTION,
11     points_selector=models.Filter(
12         must=[
13             models.FieldCondition(
14                 key="source",
15                 match=models.MatchValue(value="old_file.md"),
16             )
17         ]
18     ),
19     wait=True,
20 )

```

11 Named vectors: một point, nhiều cách biểu diễn

Named vectors dùng khi cùng một object cần nhiều vector: title vector, body vector, image vector, dense vector, sparse vector.

```

1 client.create_collection(
2     collection_name="multi_vector_docs",
3     vectors_config={
4         "title": models.VectorParams(size=384, distance=models.Distance.COSINE),
5         "body": models.VectorParams(size=384, distance=models.Distance.COSINE),
6     },
7 )

```

Upsert point có named vectors:

```

1 client.upsert(
2     collection_name="multi_vector_docs",
3     points=[
4         models.PointStruct(
5             id="doc-001",
6             vector={
7                 "title": title_vector,
8                 "body": body_vector,
9             },
10            payload={
11                "title": "Qdrant cho RAG",
12                "body": "Nội dung tài liệu...",
13            },
14        )
15     ],
16 )

```

Search theo vector title:

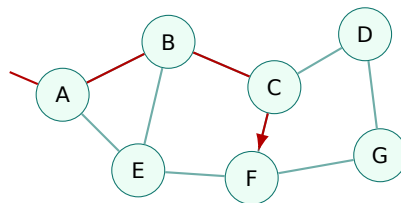
```

1 result = client.query_points(
2     collection_name="multi_vector_docs",
3     query=query_title_vector,
4     using="title",
5     limit=5,
6 )

```

12 HNSW: bản chất index tìm kiếm gần đúng

Qdrant dùng HNSW để approximate nearest neighbor search. Thay vì scan toàn bộ vector, HNSW xây một graph nhiều tầng. Khi search, thuật toán đi qua graph để tiến gần các vector tương tự.



Search đi qua graph để tới vùng vector gần query, không cần scan tất cả point

12.1 Các tham số HNSW hay gặp

Tham số	Ý nghĩa thực tế
m	Số liên kết tối đa của mỗi node trong graph. Tăng m thường tăng recall nhưng tốn RAM và index time.
ef_construct	Độ rộng search khi build index. Tăng giúp index chất lượng hơn nhưng ingest chậm hơn.
hnsw_ef	Độ rộng search lúc query. Tăng thường tăng recall nhưng latency cao hơn.
exact=True	Không dùng ANN, full scan để lấy kết quả chính xác. Dùng cho benchmark/collection nhỏ/debug.
full_scan_threshold	Ngưỡng để Qdrant quyết định full scan thay vì dùng HNSW trong một số trường hợp.

12.2 Tạo collection với HNSW config

```

1 client.create_collection(
2     collection_name="hnsw_docs",
3     vectors_config=models.VectorParams(
4         size=384,
5         distance=models.Distance.COSINE,
6     ),
7     hnsw_config=models.HnswConfigDiff(
8         m=16,
9         ef_construct=100,
10        full_scan_threshold=10_000,
11    ),

```


12)

12.3 Search với search_params

```

1 result = client.query_points(
2     collection_name="hnsw_docs",
3     query=query_vector,
4     limit=10,
5     search_params=models.SearchParams(
6         hnsw_ef=128,
7         exact=False,
8     ),
9 )

```

Cách tune nhanh

Nếu retrieval bỏ sót kết quả đúng, tăng hnsw_ef hoặc retrieve nhiều hơn rồi rerank. Nếu latency cao, giảm hnsw_ef, bật quantization, hoặc tối ưu filter/index.

13 Quantization: giảm RAM, tăng tốc, chấp nhận xấp xỉ

Quantization nén vector để giảm bộ nhớ và tăng tốc search. Scalar quantization chuyển float32 sang int8, có thể giảm RAM khoảng 4 lần. Binary quantization nén mạnh hơn, mỗi chiều thành 1 bit, có thể nhanh hơn rất nhiều nhưng mất chính xác nhiều hơn.

```

1 client.create_collection(
2     collection_name="quantized_docs",
3     vectors_config=models.VectorParams(
4         size=384,
5         distance=models.Distance.COSINE,
6     ),
7     quantization_config=models.ScalarQuantization(
8         scalar=models.ScalarQuantizationConfig(
9             type=models.ScalarType.INT8,
10            quantile=0.99,
11            always_ram=True,
12        )
13    ),
14 )

```

13.1 Giải thích tham số quantization

Tham số	Vai trò
ScalarType.INT8	Lưu vector dạng 8-bit thay vì float32.
quantile=0.99	Bỏ qua outlier cực đoan khi tính khoảng lượng tử hóa, thường giúp chất lượng ổn hơn.
always_ram=True	Giữ vector đã nén trong RAM để search nhanh.

Khi nào bật quantization?

Bật khi collection lớn, RAM bắt đầu là nút thắt hoặc latency cần tối ưu. Không nên bật ngay khi dữ liệu nhỏ và bạn đang debug chất lượng retrieval, vì quantization thêm một nguồn sai số.

14 Xây project RAG cơ bản bằng Qdrant

14.1 Cấu trúc project

```
1 qdrant_ai_lab/  
2   docker-compose.yml  
3   requirements.txt  
4   data/  
5     qdrant_notes.md  
6   src/  
7     config.py  
8     chunking.py  
9     ingest.py  
10    search.py  
11    rag.py
```

14.2 docker-compose.yml

```
1 services:  
2   qdrant:  
3     image: qdrant/qdrant:latest  
4     ports:  
5       - "6333:6333"  
6       - "6334:6334"  
7     volumes:  
8       - ./qdrant_storage:/qdrant/storage
```

14.3 requirements.txt

```
1 qdrant-client  
2 sentence-transformers  
3 numpy  
4 python-dotenv
```

14.4 src/config.py

```
1 from dataclasses import dataclass  
2  
3 @dataclass  
4 class Settings:  
5     qdrant_url: str = "http://localhost:6333"  
6     collection: str = "rag_docs"
```

```
7 embedding_model: str = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
8 vector_size: int = 384
9 top_k: int = 5
10
11 settings = Settings()
```

14.5 src/chunking.py

```
1 from typing import List
2
3
4 def chunk_text(text: str, chunk_size: int = 700, overlap: int = 120) -> List[str]:
5     """Cat text thành các chunk có overlap.
6
7     chunk_size: số ký tự tối đa mỗi chunk.
8     overlap: số ký tự lặp lại giữa 2 chunk liên tiếp để giữ ngữ cảnh.
9     """
10    if chunk_size <= overlap:
11        raise ValueError("chunk_size phải lớn hơn overlap")
12
13    chunks = []
14    start = 0
15    while start < len(text):
16        end = min(start + chunk_size, len(text))
17        chunk = text[start:end].strip()
18        if chunk:
19            chunks.append(chunk)
20            start = end - overlap
21            if start < 0:
22                start = 0
23            if end == len(text):
24                break
25    return chunks
```

14.5.1 Giải thích chunking

chunk_size quá nhỏ làm mất ngữ cảnh; quá lớn làm embedding bị nhiễu và prompt tốn token. overlap giúp câu nằm ở ranh giới không bị cắt mất nghĩa. Với tài liệu tiếng Việt, nên thử 500-1000 ký tự hoặc chunk theo heading/paragraph nếu tài liệu có cấu trúc rõ.

14.6 src/ingest.py

```
1 from pathlib import Path
2 from uuid import uuid4
3 from qdrant_client import QdrantClient, models
4 from sentence_transformers import SentenceTransformer
5
6 from config import settings
7 from chunking import chunk_text
8
9
10 def ensure_collection(client: QdrantClient) -> None:
11     if client.collection_exists(settings.collection):
12         return
13
```

```

14     client.create_collection(
15         collection_name=settings.collection,
16         vectors_config=models.VectorParams(
17             size=settings.vector_size,
18             distance=models.Distance.COSINE,
19         ),
20         on_disk_payload=True,
21     )
22
23     client.create_payload_index(
24         collection_name=settings.collection,
25         field_name="source",
26         field_schema=models.PayloadSchemaType.KEYWORD,
27     )
28     client.create_payload_index(
29         collection_name=settings.collection,
30         field_name="doc_type",
31         field_schema=models.PayloadSchemaType.KEYWORD,
32     )
33
34
35 def ingest_file(path: str, doc_type: str = "note") -> None:
36     client = QdrantClient(url=settings.qdrant_url)
37     ensure_collection(client)
38
39     model = SentenceTransformer(settings.embedding_model)
40     text = Path(path).read_text(encoding="utf-8")
41     chunks = chunk_text(text)
42     vectors = model.encode(chunks, normalize_embeddings=True).tolist()
43
44     points = []
45     for i, (chunk, vector) in enumerate(zip(chunks, vectors)):
46         points.append(
47             models.PointStruct(
48                 id=str(uuid4()),
49                 vector=vector,
50                 payload={
51                     "text": chunk,
52                     "source": Path(path).name,
53                     "chunk_id": i,
54                     "doc_type": doc_type,
55                 },
56             )
57         )
58
59     client.upsert(
60         collection_name=settings.collection,
61         points=points,
62         wait=True,
63     )
64     print(f"Ingested {len(points)} chunks from {path}")
65
66
67 if __name__ == "__main__":
68     ingest_file("data/qdrant_notes.md", doc_type="qdrant")

```

14.7 src/search.py

```

1 from typing import Optional, List, Dict, Any
2 from qdrant_client import QdrantClient, models
3 from sentence_transformers import SentenceTransformer

```

```
4
5 from config import settings
6
7
8 class QdrantSearcher:
9     def __init__(self):
10         self.client = QdrantClient(url=settings.qdrant_url)
11         self.model = SentenceTransformer(settings.embedding_model)
12
13     def search(
14         self,
15         query: str,
16         top_k: int = settings.top_k,
17         source: Optional[str] = None,
18         doc_type: Optional[str] = None,
19     ) -> List[Dict[str, Any]]:
20         query_vector = self.model.encode(query, normalize_embeddings=True).tolist()
21
22         must = []
23         if source:
24             must.append(models.FieldCondition(
25                 key="source",
26                 match=models.MatchValue(value=source),
27             ))
28         if doc_type:
29             must.append(models.FieldCondition(
30                 key="doc_type",
31                 match=models.MatchValue(value=doc_type),
32             ))
33
34         flt = models.Filter(must=must) if must else None
35
36         response = self.client.query_points(
37             collection_name=settings.collection,
38             query=query_vector,
39             query_filter=flt,
40             limit=top_k,
41             with_payload=True,
42             with_vectors=False,
43             search_params=models.SearchParams(hnsw_ef=128, exact=False),
44         )
45
46         return [
47             {
48                 "id": hit.id,
49                 "score": hit.score,
50                 "text": hit.payload.get("text"),
51                 "source": hit.payload.get("source"),
52                 "chunk_id": hit.payload.get("chunk_id"),
53             }
54             for hit in response.points
55         ]
56
57
58 if __name__ == "__main__":
59     searcher = QdrantSearcher()
60     hits = searcher.search("payload index trong Qdrant dùng để làm gì?")
61     for h in hits:
62         print(round(h["score"], 4), h["source"], h["chunk_id"])
63         print(h["text"][:300])
64         print("---")
```

14.8 src/rag.py

```

1 from search import QdrantSearcher
2
3
4 def build_prompt(question: str, contexts: list[dict]) -> str:
5     context_text = "\n\n".join(
6         f"[Nguon: {c['source']} | chunk {c['chunk_id']} | score
7         ↪ {c['score']:.3f}]\n{c['text']}"
8         for c in contexts
9     )
10
11     return f"""
12 Ban la tro ly AI. Hay tra loi dua tren CONTEXT.
13 Neu context khong du, hay noi ro la chua du thong tin.
14
15 CONTEXT:
16 {context_text}
17
18 QUESTION:
19 {question}
20
21 ANSWER:
22 """.strip()
23
24 def retrieve_for_rag(question: str) -> tuple[str, list[dict]]:
25     searcher = QdrantSearcher()
26     contexts = searcher.search(question, top_k=5)
27     prompt = build_prompt(question, contexts)
28     return prompt, contexts
29
30
31 if __name__ == "__main__":
32     prompt, contexts = retrieve_for_rag("Qdrant collection va point khac nhau the nao?")
33     print(prompt)

```

14.9 Luồng input/output của RAG

```

1 Question:
2   "Payload index trong Qdrant dùng để làm gì?"
3
4 Embedding query:
5   [0.02, -0.11, ..., 0.08]
6
7 Qdrant returns top-k:
8   hit 1: score 0.84, source qdrant_notes.md, chunk 12
9   hit 2: score 0.77, source qdrant_notes.md, chunk 4
10
11 Prompt to LLM:
12   CONTEXT: hit texts...
13   QUESTION: Payload index...
14
15 Final answer:
16   Payload index giúp Qdrant filter nhanh hơn...

```

15 Hybrid search: dense + sparse + fusion

Dense vector hiểu nghĩa, nhưng đôi khi kém với mã sản phẩm, tên thuốc, tên hàm, số hiệu văn bản. Sparse/keyword search giải exact match. Hybrid search kết hợp cả hai.

15.1 Khi nào cần hybrid?

- Người dùng có thể hỏi mơ hồ: “cách tăng tốc truy vấn Qdrant”.
- Người dùng có thể hỏi chính xác: “hnsw_ef”, “XR-7742”, “Điều 12 khoản 3”.
- Tài liệu có nhiều thuật ngữ kỹ thuật, mã lỗi, tên hàm, tên biến.

15.2 Universal Query API với RRF

Ví dụ dưới đây dùng `models.Document`, yêu cầu cài `qdrant-client[fastembed]` hoặc dùng Qdrant Cloud Inference tùy môi trường. Ý tưởng chính: prefetch dense và sparse, sau đó `FusionQuery` gộp rank.

```

1 from qdrant_client import QdrantClient, models
2
3 client = QdrantClient(url="http://localhost:6333")
4 query = "cách cấu hình HNSW parameters"
5
6 result = client.query_points(
7     collection_name="hybrid_docs",
8     prefetch=[
9         models.Prefetch(
10             query=models.Document(
11                 text=query,
12                 model="sentence-transformers/all-MiniLM-L6-v2",
13             ),
14             using="dense",
15             limit=20,
16         ),
17         models.Prefetch(
18             query=models.Document(
19                 text=query,
20                 model="Qdrant/bm25",
21             ),
22             using="sparse",
23             limit=20,
24         ),
25     ],
26     query=models.FusionQuery(fusion=models.Fusion.RRF),
27     limit=10,
28     with_payload=True,
29 )

```

15.3 RRF trực quan

Doc	Rank dense	Rank sparse	Ý nghĩa
-----	------------	-------------	---------

D1	1	3	Vừa gần nghĩa vừa có từ khóa liên quan, rất đáng tin.
D2	2	4	Tốt, nhưng kém D1.
D3	7	1	Exact keyword mạnh, semantic yếu hơn.
D4	1	không có	Có thể gần nghĩa nhưng thiếu keyword quan trọng.

RRF không so sánh trực tiếp score giữa dense và sparse vì hai score scale khác nhau. Nó gộp theo thứ hạng. Đây là lý do hybrid search ổn định hơn việc cộng raw score.

16 Multivector và ColBERT reranking

Multivector cho phép một point chứa ma trận vector thay vì một vector duy nhất. Mô hình late interaction như ColBERT tạo vector cho từng token/đoạn nhỏ, sau đó so khớp chi tiết hơn.

```

1 client.create_collection(
2     collection_name="colbert_docs",
3     vectors_config=models.VectorParams(
4         size=128,
5         distance=models.Distance.COSINE,
6         multivector_config=models.MultiVectorConfig(
7             comparator=models.MultiVectorComparator.MAX_SIM,
8         ),
9     ),
10 )

```

16.1 Khi nào dùng multivector?

- Bạn cần reranking chính xác hơn sau khi đã retrieve candidate bằng dense/sparse.
- Tài liệu dài, câu hỏi cần khớp nhiều chi tiết nhỏ.
- Bạn chấp nhận chi phí lưu trữ và search cao hơn vector đơn.

17 Multitenancy và phân quyền retrieval

Không nên tạo một collection cho mỗi user nếu có nhiều user. Cách thường dùng là một collection chung, mỗi point có tenant_id hoặc user_id, sau đó filter bắt buộc ở mọi query.

```

1 client.create_payload_index(
2     collection_name="agent_memories",
3     field_name="tenant_id",
4     field_schema=models.KeywordIndexParams(
5         type=models.KeywordIndexType.KEYWORD,
6         is_tenant=True,
7     ),
8 )

```


Search memory của một tenant:

```

1 tenant_filter = models.Filter(
2     must=[
3         models.FieldCondition(
4             key="tenant_id",
5             match=models.MatchValue(value="user_001"),
6         )
7     ]
8 )
9
10 result = client.query_points(
11     collection_name="agent_memories",
12     query=query_vector,
13     query_filter=tenant_filter,
14     limit=5,
15 )

```

Không bao giờ chỉ filter ở phía UI

Với dữ liệu nhiều người dùng, filter tenant_id/access_roles phải nằm trong truy vấn Qdrant hoặc tầng backend bắt buộc. Nếu chỉ lọc ở frontend, dữ liệu nhạy cảm có thể bị lộ.

18 Ứng dụng Qdrant cho memory của agent

18.1 Memory nên lưu gì?

Không phải mọi tin nhắn đều đáng lưu dài hạn. Nên lưu các sự kiện, sở thích, quyết định, mục tiêu, profile dự án, lỗi đã gặp, giải pháp đã thử. Không nên lưu rác hội thoại hoặc thông tin quá nhạy cảm nếu không có lý do rõ ràng.

Field	Vai trò trong memory
text	Nội dung memory đã chuẩn hóa, ví dụ "User đang học Qdrant để xây AI memory".
memory_type	profile, preference, project, bug, solution, fact, task.
importance	Điểm quan trọng 1-5 hoặc low/medium/high.
created_at	Thời điểm lưu.
last_accessed	Phục vụ decay/refresh.
tenant_id/session_id	Tách người dùng hoặc phiên.
source	Nguồn: chat, uploaded_file, manual_note.

18.2 Code memory service tối giản

```

1 from datetime import datetime, timezone
2 from uuid import uuid4
3 from qdrant_client import QdrantClient, models
4 from sentence_transformers import SentenceTransformer
5
6
7 class VectorMemory:

```

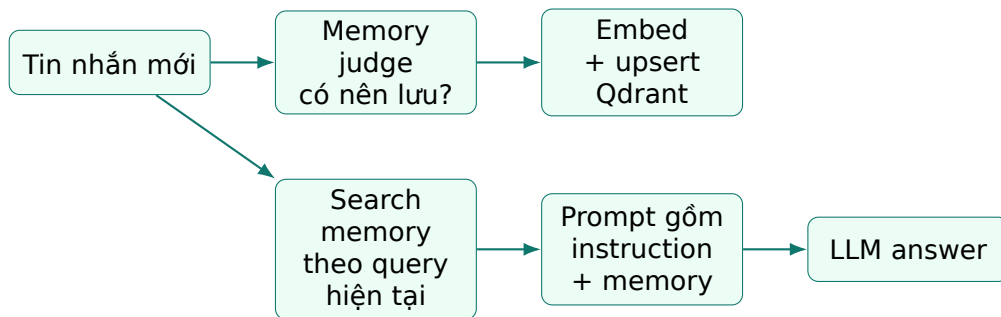
```
8 def __init__(self, url: str, collection: str, model_name: str):
9     self.client = QdrantClient(url=url)
10    self.collection = collection
11    self.model = SentenceTransformer(model_name)
12
13    def create_if_needed(self, dim: int = 384):
14        if self.client.collection_exists(self.collection):
15            return
16        self.client.create_collection(
17            collection_name=self.collection,
18            vectors_config=models.VectorParams(
19                size=dim,
20                distance=models.Distance.COSINE,
21            ),
22            on_disk_payload=True,
23        )
24        for field in ["tenant_id", "memory_type", "source"]:
25            self.client.create_payload_index(
26                collection_name=self.collection,
27                field_name=field,
28                field_schema=models.PayloadSchemaType.KEYWORD,
29            )
30
31    def add_memory(
32        self,
33        tenant_id: str,
34        text: str,
35        memory_type: str = "fact",
36        importance: int = 3,
37        source: str = "chat",
38    ) -> str:
39        vector = self.model.encode(text, normalize_embeddings=True).tolist()
40        point_id = str(uuid4())
41        now = datetime.now(timezone.utc).isoformat()
42
43        self.client.upsert(
44            collection_name=self.collection,
45            points=[
46                models.PointStruct(
47                    id=point_id,
48                    vector=vector,
49                    payload={
50                        "tenant_id": tenant_id,
51                        "text": text,
52                        "memory_type": memory_type,
53                        "importance": importance,
54                        "source": source,
55                        "created_at": now,
56                    },
57                )
58            ],
59            wait=True,
60        )
61        return point_id
62
63    def search_memory(self, tenant_id: str, query: str, top_k: int = 5):
64        query_vector = self.model.encode(query, normalize_embeddings=True).tolist()
65        flt = models.Filter(
66            must=[
67                models.FieldCondition(
68                    key="tenant_id",
69                    match=models.MatchValue(value=tenant_id),
70                )
71            ]
72        )
```

```

73     response = self.client.query_points(
74         collection_name=self.collection,
75         query=query_vector,
76         query_filter=flt,
77         limit=top_k,
78         with_payload=True,
79     )
80     return response.points

```

18.3 Cơ chế memory retrieval trong agent



19 Đánh giá chất lượng retrieval

Không nên chỉ nhìn cảm tính. Cần tạo bộ test query và ground truth.

19.1 Các metric quan trọng

Metric	Ý nghĩa
Recall@k	Trong top-k có lấy được tài liệu đúng không. Quan trọng nhất cho stage retrieval.
MRR	Tài liệu đúng xuất hiện càng sớm càng tốt.
Precision@k	Trong top-k có bao nhiêu kết quả thật sự liên quan.
Latency P50/P95	Thời gian search trung vị và đuôi chậm. Production cần nhìn P95/P99.
Hit rate	Tỷ lệ query có ít nhất một kết quả vượt ngưỡng.

19.2 Code đánh giá Recall@k đơn giản

```

1 def recall_at_k(search_fn, eval_set: list[dict], k: int = 5) -> float:
2     """eval_set item: {"query": str, "relevant_ids": set[str]}"""
3     ok = 0
4     for item in eval_set:
5         hits = search_fn(item["query"], top_k=k)
6         retrieved_ids = {str(h["id"]) for h in hits}
7         if retrieved_ids & set(item["relevant_ids"]):
8             ok += 1
9     return ok / len(eval_set)

```

20 Tối ưu production checklist

Vấn đề	Hướng xử lý
Search chậm	Tạo payload index, giảm limit, tune hnsw_ef, dùng gRPC, bật quantization, kiểm tra filter cardinality.
Recall thấp	Tăng hnsw_ef, tăng candidate top-k, cải thiện chunking, đổi embedding model, thêm reranker/hybrid.
Tốn RAM	on_disk_payload, quantization, vectors on disk, shard hợp lý, xóa payload dư.
Dữ liệu nhiều tenant	Một collection chung + tenant_id filter + tenant index. Không tạo collection/user nếu user rất nhiều.
Cần backup	Dùng snapshot collection hoặc full snapshot, test restore định kỳ.
Cần phân quyền	Payload access_roles/tenant_id, filter bắt buộc ở backend, không tin frontend.
Data update liên tục	Batch upsert, wait=False cho ingest lớn nếu chấp nhận eventual visibility, theo dõi optimizer.

21 Những lỗi phổ biến khi mới dùng Qdrant

1. Sai vector size

Collection tạo size=384 nhưng model trả 768. Cách sửa: kiểm tra len(embedding) trước khi tạo collection hoặc tạo lại collection đúng dimension.

2. Lưu cả tài liệu dài thành một vector

Một PDF dài 50 trang thành một vector sẽ làm retrieval rất tệ. Hãy chunk theo heading/paragraph/page và lưu metadata nguồn.

3. Không dùng payload filter cho tenant/quyền

RAG nội bộ rất dễ rò rỉ dữ liệu nếu retrieval không filter quyền từ đầu.

4. Chỉ dùng vector search cho query chứa mã/keyword

Với mã lỗi, số hiệu, tên hàm, thuốc, điều khoản pháp lý, nên cân nhắc hybrid search.

22 Bài tập vận dụng

22.1 Mức cơ bản

1. Tạo collection student_notes dùng model 384 chiều, metric cosine.
2. Upsert 20 đoạn ghi chú về AI, mỗi point có payload text, topic, source.
3. Query: “vector database dùng để làm gì?” và in top-5 kết quả.
4. Thêm filter topic = rag. So sánh kết quả có và không filter.

22.2 Mức trung bình

- 1. Tạo payload index cho topic, source, created_at.
- 2. Viết hàm search(query, topic=None, source=None, top_k=5).
- 3. Viết hàm delete_by_source(source) để xóa toàn bộ chunk của một file cũ.
- 4. Tạo script ingest nhiều file .md trong folder data/.

22.3 Mức nâng cao

- 1. Xây RAG mini: ingest tài liệu, retrieve top-k, build prompt cho LLM local/API.
- 2. Tạo eval set 20 query, mỗi query có relevant chunk id, đo Recall@3 và Recall@5.
- 3. Tune hnsw_ef: 32, 64, 128, 256. Ghi lại latency và Recall@5.
- 4. Chuyển sang hybrid search dense+sparse, so sánh với dense-only trên query có keyword kỹ thuật.
- 5. Xây VectorMemory cho agent: add memory, search memory theo tenant, xóa memory theo type.

22.4 Đề tài project cuối khóa

AI Academic Reading Memory

Xây một agent đọc paper: ingest PDF/paper notes vào Qdrant, dùng hybrid search để tìm đoạn liên quan, dùng memory để lưu các khái niệm người học hay hỏi, có dashboard log query/hit/score/latency. Mục tiêu: trả lời câu hỏi học thuật có dẫn nguồn chunk, đo Recall@5 và P95 latency.

23 Lộ trình luyện tập 14 ngày

Ngày	Việc cần làm
1	Cài Docker, chạy Qdrant, mở dashboard.
2	Tạo collection, upsert vector giả để hiểu point/payload.
3	Dùng sentence-transformers tạo embedding thật.
4	Search cơ bản, phân tích score và top-k.
5	Filter với must/should/must_not, range, nested field.
6	Payload index, scroll, retrieve, update, delete.
7	Viết ingest nhiều file markdown.
8	Viết search module và build prompt RAG.
9	Thiết kế metadata schema cho project thật của bạn.
10	Benchmark chunk size, top-k, score threshold.
11	Học HNSW: tune hnsw_ef, m, ef_construct.
12	Hybrid search dense+sparse.
13	Memory service cho agent.
14	Đánh giá Recall@k, MRR, latency; viết báo cáo kết quả.

24 Cheat sheet lệnh Qdrant/Python hay dùng

Lệnh	Mục đích
<code>QdrantClient(url=...)</code>	Kết nối server/cloud.
<code>QdrantClient(":memory:")</code>	Local test trong RAM.
<code>client.collection_exists(name)</code>	Kiểm tra collection.
<code>client.create_collection(...)</code>	Tạo collection.
<code>client.delete_collection(name)</code>	Xóa collection.
<code>client.upsert(...)</code>	Thêm/cập nhật point.
<code>client.query_points(...)</code>	Search/recommend/discover/hybrid qua Universal Query API.
<code>client.retrieve(...)</code>	Lấy point theo id.
<code>client.scroll(...)</code>	Duyệt point theo trang/filter.
<code>client.set_payload(...)</code>	Cập nhật một phần payload.
<code>client.overwrite_payload(...)</code>	Ghi đè toàn bộ payload của point.
<code>client.delete(...)</code>	Xóa point theo id hoặc filter.
<code>client.create_payload_index(...)</code>	Tạo index cho field metadata.
<code>models.Filter(...)</code>	Biểu diễn điều kiện filter.
<code>models.SearchParams(...)</code>	Điều chỉnh search như <code>hnsw_ef</code> , <code>exact</code> .
<code>models.Prefetch(...)</code>	Tạo stage candidate cho hybrid/multistage query.
<code>models.FusionQuery(...)</code>	Gộp nhiều prefetch bằng RRF/DBSF.

25 Tài liệu tham khảo chính

- Qdrant Documentation: <https://qdrant.tech/documentation/>
- Local Quickstart: <https://qdrant.tech/documentation/quickstart/>
- Collections: <https://qdrant.tech/documentation/manage-data/collections/>
- Filtering: <https://qdrant.tech/documentation/search/filtering/>
- Search API: <https://qdrant.tech/documentation/search/search/>
- Python Client API: https://python-client.qdrant.tech/qdrant_client.qdrant_client
- Hybrid Queries: <https://qdrant.tech/documentation/search/hybrid-queries/>
- Qdrant Essentials - Hybrid Search and Universal Query API: <https://qdrant.tech/course/essentials/day-3/hybrid-search/>
- Vectors and Multivectors: <https://qdrant.tech/documentation/manage-data/vectors/>
- Quantization: <https://qdrant.tech/documentation/manage-data/quantization/>
- Multitenancy: <https://qdrant.tech/documentation/manage-data/multitenancy/>